

高级语言程序设计

实验报告

南开大学 计算机大类

姓名 孙志航

学号 2513842

班级 1009

2026 年 5 月 13 日

目录

高级语言程序设计大作业实验报告	1
一. 作业题目	1
二. 开发软件	1
三. 课题要求	1
四. 主要流程	1
1. 整体流程.....	1
(1) 实现思路.....	1
(2) 项目结构与类设计.....	4
2. 主要算法或公式.....	5
(1) 作物成熟判定.....	6
(2) 作物收益与成本.....	6
(3) 祈福随机收益.....	6
(4) 求雨范围效果.....	6
(5) 按钮冷却机制.....	7
(6) 土地网格布局.....	7
(7) 货币与奖杯.....	7
3. 单元测试.....	8
五. 单元测试	8
六. 收获	9
1. 信号槽与对象解耦.....	9
2. 事件驱动模型.....	10
3. 资源系统与冷却实现.....	10
七. AI 工具使用说明	10

高级语言程序设计大作业实验报告

一. 作业题目

My Farm —— 农场模拟游戏

二. 开发软件

项目语言: C++

图形化平台: Qt (Qt Widgets、Qt Multimedia)

IDE: Qt Creator 4.11.1 (Community)

构建套件: Desktop Qt 5.14.2 MinGW 32-bit

构建模式: Debug

三. 课题要求

- 1) 面向对象
- 2) 单元测试
- 3) 模型部分
- 4) 验证

四. 主要流程

1. 整体流程

(1) 实现思路:

程序从 main.cpp 启动, 创建 QApplication 与 MainWindow。MainWindow 以 GameCanvasWidget 作为中央部件铺设主背景, 并在其上叠放 3 行 5 列共 15 块 FarmLandWidget 土地、货币标签、奖杯标签、悬浮信息标签、雨景遮罩, 以及左下角祈福、右下角求雨两个 ImageButton, 构成 1920×1080 的游戏主界面。

用户点击任一土地时，FarmLandWidget 通过 landClicked 信号通知 MainWindow，后者弹出 ActionDialog 操作框。ActionDialog 根据该格作物的当前状态（空地 / 未成熟 / 已成熟）动态生成不同按钮：空地显示“小麦”“水稻”种植按钮；未成熟显示“浇水”“施肥”“铲除”；成熟则显示“卖掉”“铲除”。按钮回调 MainWindow 中的核心函数修改 FarmLandWidget 的内部状态，并刷新货币与界面。

祈福与求雨两个全局按钮通过自定义的 ImageButton 实现冷却机制：点击后启动 QTimer 倒计时，冷却中按钮半透明并屏蔽点击。求雨事件还会在画布上叠加 rain_scene.png 雨景图层、播放雨声音效，并对所有有作物的土地浇水次数 +1；祈福事件随机获得 0~\$100 货币。

整个程序由事件驱动：鼠标点击、悬浮、按钮 clicked 信号、QTimer::timeout 触发状态变更，最终通过 QWidget::update() 调用各组件的 paintEvent() 完成画面刷新。主流程如图 1 所示。

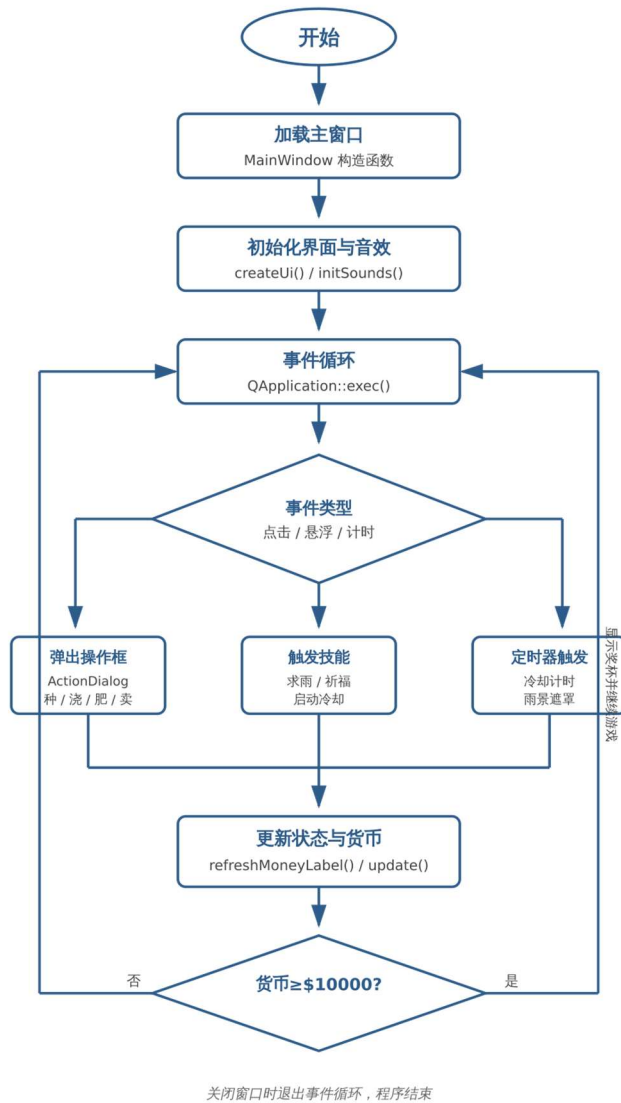


图 1 程序整体流程图

游戏主界面布局如图 2 所示。

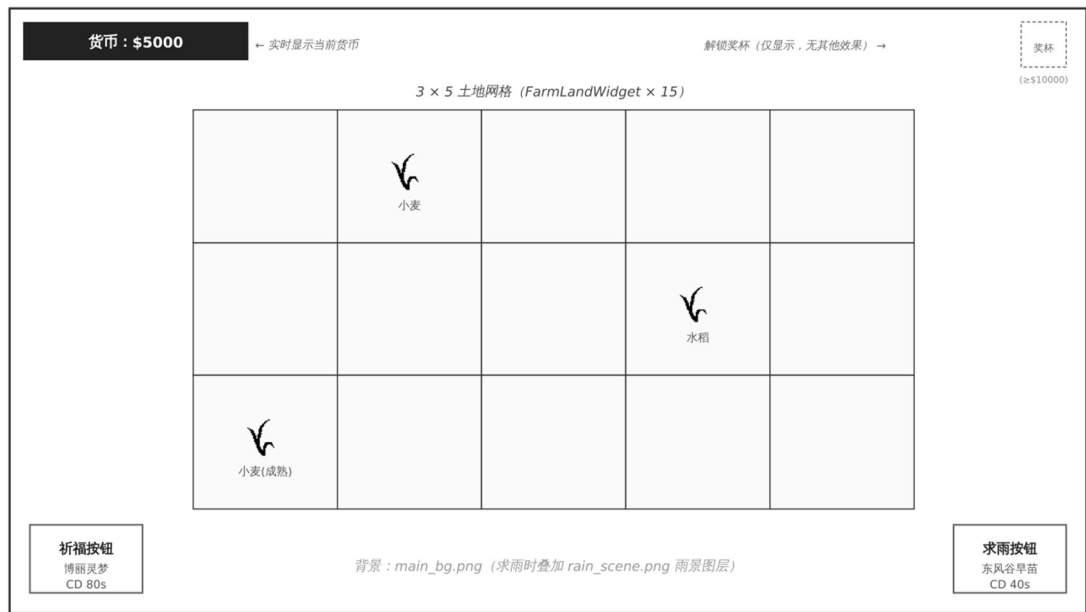


图 2 游戏主界面布局示意图

(2) 项目结构与类设计:

项目按职责拆分为窗口与画布、土地与作物、操作弹窗、可冷却按钮、公共类型五个模块，各文件职责如表 1 所示。

文件	主要职责
main.cpp	程序入口；创建 QApplication 与 MainWindow，设置窗口图标后进入事件循环
mainwindow.h / .cpp	顶层调度类；管理货币、奖杯、土地数组、技能按钮、音效，并实现 plantCrop / waterCrop / fertilizeCrop / removeCrop / sellCrop 等核心逻辑
gamecanvaswidget.h / .cpp	中央画布；重写 paintEvent() 绘制主背景 main_bg.png
farmlandwidget.h / .cpp	单格土地控件；封装作物类型、浇水次数、施肥次数；负责绘制作物图与上报点击 / 悬浮信号
actiondialog.h / .cpp	土地操作弹窗；根据土地状态动态生成种植 / 培养 / 卖出 / 铲除按钮
imagebutton.h / .cpp	继承 QPushButton 的可冷却按钮；用 QDateTime + QTimer + QGraphicsOpacityEffect 实现冷却倒计时与视觉反馈
gametypes.h	公共类型；定义作物枚举 CropType 与 cropName() 工具函数

文件	主要职责
resources/ 与 resources.qrc	游戏所用图片与音频资源，通过 Qt 资源系统打包进可执行文件

表 1 项目文件与职责

类之间的关系如下：MainWindow 继承 QMainWindow，作为顶层调度者持有 1 个 GameCanvasWidget、15 个 FarmLandWidget（3 行 × 5 列网格）、2 个 ImageButton（祈福、求雨），并按需打开 AlertDialog。GameCanvasWidget、FarmLandWidget 均继承自 QWidget；ImageButton 继承自 QPushButton；AlertDialog 继承自 QDialog。FarmLandWidget 不持有 MainWindow 指针，而是通过 landClicked、hoverInfoRequested、hoverLeft 三个信号与 MainWindow 解耦，这样土地控件本身只关心自己的数据与绘制，符合“高内聚、低耦合”的设计原则。完整类图如图 3 所示。

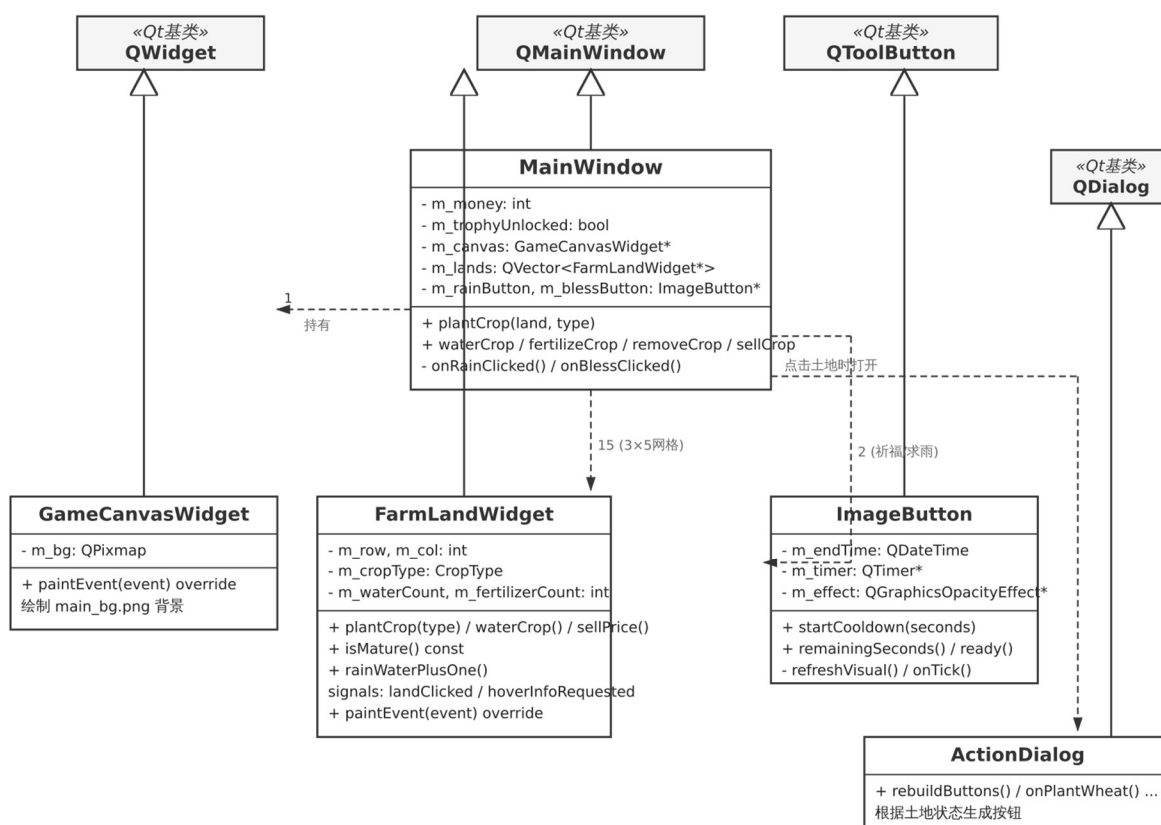


图 3 项目类图

2. 主要算法或公式

(1) 作物成熟判定

作物的成熟条件为浇水次数 ≥ 10 且施肥次数 ≥ 5 ，两个条件必须同时满足：
`FarmLandWidget::isMature()` 在 `ActionDialog` 重建按钮时被调用，用于决定按钮组合。核心代码如下：

```
bool FarmLandWidget::isMature() const
{
    if (!hasCrop()) return false;
    return m_waterCount >= 10 && m_fertilizerCount >= 5;
}
```

(2) 作物收益与成本

两种作物的经济模型如表 2 所示。水稻一次净收益更高，但需要更高初始资金，因此初始阶段适合先种小麦累积资金，后期再切换到水稻提高单格产出。

作物	种植成本	卖出收益	净利润	利润率
小麦	\$100	\$150	\$50	50%
水稻	\$200	\$300	\$100	50%

表 2 两种作物的经济参数

(3) 祈福随机收益

点击祈福按钮（博丽灵梦）后，每次随机获得 $[0, 100]$ 内的整数货币，冷却 80 秒。使用 Qt 自带的 `QRandomGenerator` 全局实例生成均匀分布的随机数：

```
int addMoney = QRandomGenerator::global()->bounded(101); // 等概率返回 0~100
m_money += addMoney;
refreshMoneyLabel();
```

`QRandomGenerator::bounded(101)` 的取值范围为 $[0, 101)$ ，即 $0 \sim 100$ 共 101 个等可能整数。期望收益约为 50 元，与小麦的单次净利润相当，但冷却长达 80 秒，因此祈福只是辅助手段，不能替代正常的种植循环。

(4) 求雨范围效果

点击求雨按钮（东风谷早苗）后，对全部 15 块土地遍历，凡是有作物的地块浇水次数 +1，并在画布上叠加雨景图层与播放雨声；冷却 40 秒。关键代码如下：

```
int affected = 0;
for (FarmLandWidget *land : m_lands)
{
```



```

        if (land->hasCrop())
        {
            land->rainWaterPlusOne();
            ++affected;
        }
    }
}

```

求雨等价于一次性给所有有作物的地块“免费浇水”，理论最大收益为 15 次浇水。考虑到单次浇水操作需要点击土地 → 弹窗 → 选择浇水三步，求雨在中后期阶段能显著节省操作。

（5）按钮冷却机制

ImageButton 通过 QDateTime + QTimer + QGraphicsOpacityEffect 三件组合实现冷却。调用 startCooldown(seconds) 时记录终点时间，QTimer 每秒触发一次更新剩余秒数。剩余秒数计算公式：剩余秒数 = QDateTime().secsTo(m_endTime)。冷却中按钮的不透明度被设为 0.55，鼠标光标变为禁止图标，点击事件被拦截并以 QToolTip 提示剩余时间。

```

void ImageButton::startCooldown(int seconds)
{
    if (seconds <= 0) return;
    m_endTime = QDateTime::currentDateTime().addSecs(seconds);
    m_timer->start(1000);
    refreshVisual();
    setToolTip(statusText());
}

```

这种实现的好处是即使界面被遮挡或切到后台，倒计时仍然以“真实时间”为准，不会因为定时器延迟丢失精度。

（6）土地网格布局

3 行 5 列的土地网格起点为 (330, 177)，总尺寸为 1280 × 723 像素。单格尺寸通过整除计算：

```

cellW = 1280 / 5 = 256
cellH = 723 / 3 = 241
// 最后一列宽度 = 1280 - 256 * 4 = 256
// 最后一行高度 = 723 - 241 * 2 = 241

```

最后一行与最后一列特别使用剩余像素而不是 cellW、cellH，避免因为整除产生的累积误差使网格右下角与背景图错位。

（7）货币与奖杯

初始货币 \$5000。refreshMoneyLabel() 在每次货币变更时被调用，若 m_money $\geq$ 10000 则永久解锁奖杯 (m_trophyUnlocked = true)，右上角的 trophy.png 图标常驻显示。一旦解锁就不会因为后续花钱降到 10000 以下而隐藏，符合“成就一次即永久”的常识。

3. 单元测试

围绕以下几个方面设计测试用例：界面是否能正确加载、土地点击是否能弹出操作框、不同状态下按钮组合是否正确、货币是否被正确扣减或增加、未成熟作物是否无法卖出、求雨是否对全部有作物的地块生效、按钮冷却中是否正确屏蔽点击、奖杯是否在货币达到阈值时显示。

五. 单元测试

测试用例如表 3 所示。

编号	测试内容	预期结果	结果
1	启动程序	主界面正常显示，背景、15 块土地、货币 \$5000、祈福、求雨按钮均可见	通过
2	将鼠标悬浮到任一土地	出现悬浮信息框，显示作物状态（空地或具体作物的浇水 / 施肥次数）	通过
3	点击空地	弹出 AlertDialog，仅显示“小麦 \$100”“水稻 \$200”两个种植按钮	通过
4	在 \$5000 余额下选择种植小麦	货币扣减为 \$4900，土地变为未成熟小麦，悬浮信息显示浇水 0、施肥 0	通过
5	把货币花到 \$50 后再种水稻 \$200	弹出“货币不足”提示，土地保持空地	通过
6	点击未成熟作物	弹出对话框显示“浇水”“施肥”“铲除”三个按钮	通过
7	浇水 10 次、施肥 5 次后再点击该地	作物变为成熟态；弹出对话框显示“卖掉 获得 \$XX”与“铲除”	通过
8	卖出成熟小麦 / 水稻	货币分别 +\$150 / +\$300，土地恢复为空地	通过

编号	测试内容	预期结果	结果
9	浇水 9 次、施肥 5 次（未达成熟）	点击土地仍显示培养按钮组合，无“卖掉”选项	通过
10	点击祈福按钮	货币随机增加 0~\$100，按钮变半透明并显示剩余 CD；80 秒内再次点击无效	通过
11	在多块地有作物时点击求雨	画面叠加雨景图层、播放雨声；提示有 N 片作物被浇水；冷却 40 秒	通过
12	无作物时点击求雨	仍正常播放雨景，但提示“0 片作物得到滋润”，无空指针或崩溃	通过
13	点击铲除	作物消失，土地恢复为空地，不返还任何货币	通过
14	通过种植、卖出累计货币达到 \$10000	右上角奖杯图标永久显示；继续花钱降到 \$10000 以下后奖杯仍保留	通过
15	快速连续点击同一土地	只能弹出一个 QDialogBox；关闭后再次点击才会重新弹出，不会出现叠加窗口	通过

表 3 测试用例

测试过程中重点关注以下三个易错点。第一，未成熟作物绝不能进入“卖掉”分支：在 `ActionDialog::rebuildButtons()` 中通过 `isMature()` 严格分流按钮组合；同时 `MainWindow::sellCrop()` 内部再次校验 `sellPrice()` 是否 > 0 ，做了双重保险。第二，求雨在空地图上调用 `land->hasCrop()` 应返回 `false` 而非崩溃：因此 `rainWaterPlusOne()`、`waterCrop()`、`fertilizeCrop()` 都先做 `hasCrop()` 判断。第三，冷却中按钮被点击时不能进入对应槽函数：`ImageButton::mousePressEvent()` 在 `ready()` 为 `false` 时直接 `accept` 事件并提示，使得槽函数不会触发。

六. 收获

1. 信号槽与对象解耦

最初我打算让 `FarmLandWidget` 直接持有 `MainWindow` 指针来回调种植、浇水等函数，但很快意识到这会让 `FarmLandWidget` 强依赖于 `MainWindow`，难以单独使用与测试。改用信号槽以后，`FarmLandWidget` 只负责发出 `landClicked` 信号，由 `MainWindow` 决定如何响应，两者通过 Qt 的元对象系统在运行期连接，类之间彻底解

耦。这让我对“依赖倒置”与“关注点分离”有了具体的体会。

2. 事件驱动模型

Qt 程序并不是按顺序从头跑到尾，而是由 `QApplication::exec()` 进入事件循环，等待外部输入。每一次鼠标点击、按钮 `clicked`、`QTimer::timeout` 都对应一个事件，被分派到相应槽函数；槽函数只负责改状态，最后通过 `update()` 触发 `paintEvent()` 重绘。理解这一点之后，调试时不再纠结“为什么这个变量改完没生效”——答案往往是漏调用了 `update()`。

3. 资源系统与冷却实现

项目初期我直接用绝对路径加载图片，结果换电脑就跑不起来。学习并使用 Qt 的 `.qrc` 资源系统后，所有图片和音频被打包进可执行文件，跨设备运行无需带资源文件夹。冷却按钮一开始我准备用一个简单的 `int` 变量自减，但发现窗口最小化或卡顿时倒计时会失真；改用 `QDateTime::addSecs()` 记录终点时间、`QTimer` 周期刷新显示后，冷却时间与真实流逝时间始终一致。这个例子让我理解到：用“时间戳”表示状态，比用“累加计数”表示状态更稳健。

七. AI 工具使用说明

AI 工具名称及版本：ChatGPT Images 2.0

具体用途：生成部分游戏资源图片

使用位置：`/resources/`文件夹中的部分图片